Numbers Every Engineer Must Know

System Design Fundamentals — Latency · Scale · Storage · Back-of-Envelope Math

Kashyap Pandey | Principal / Staff SDE Prep | March – May 2026

WHY THESE NUMBERS MATTER

In a system design interview, every component choice you make must be backed by numbers. 'I will use Redis' is a Senior Engineer answer. 'I will use Redis because DB queries cost 50ms and our SLA is 20ms, so I need sub-millisecond cache reads' is a Principal Engineer answer. These numbers are the evidence behind every decision.

The 3 questions these numbers answer:

- **WHAT to use?** — Latency numbers tell you which component fits your SLA.
- **WHEN to scale?** — Scale thresholds tell you which solution to add at which traffic level.
- **HOW MUCH storage?** — Storage math tells you if you need one server or a distributed cluster.

SECTION 1 — LATENCY NUMBERS

Latency = how long an operation takes to complete. The key insight: raw hardware speed and real-world query speed are VERY different. The software layer on top of hardware is where time gets spent.

The Critical Distinction: Hardware Speed vs Real-World Speed

This was corrected from the original session. Here is the accurate picture with full explanation:

Why is SSD 0.1ms but a database query 10-50ms if the database sits on an SSD?

Raw SSD hardware: reading bytes off the physical chip takes 0.1ms. That is the hardware doing one thing — fetch bytes.

A database query on the same SSD: the DB engine must parse your SQL, plan the query, find and walk the index (B-tree), fetch the matching pages from disk, filter rows, join tables if needed, sort results, and return them. Each of those steps adds time.

Example: SELECT * FROM tweets WHERE user_id = 77291 ORDER BY created_at DESC LIMIT 20

→ Parse SQL: ~0.01ms
→ Check query plan cache: ~0.01ms
→ Walk B-tree index on user_id: ~1-5ms (may need multiple disk reads)
→ Fetch 20 matching rows from disk: ~1-10ms
→ Sort by created_at: ~0.1ms
→ Network back to app server: ~1ms
→ TOTAL: 10-50ms depending on index quality, data size, server load

The hardware is fast. The software on top of it is where time is spent. This is why cache exists.

Operation	Raw Hardware	Real-World (with software)	Why the Gap?
L1 Cache (CPU chip)	**0.0005ms**	**0.0005ms**	No software layer. Pure hardware. Rarely relevant in system design.
RAM read (Cache hit e.g. Redis)	**0.0001ms**	**0.1-1ms**	Redis adds network hop (~0.5ms) + protocol parsing. Still very fast.
SSD read (raw hardware)	**0.1ms**	**10-50ms**	Database adds: SQL parsing, query planning, index walk, row filtering, network.
HDD read (old spinning disk)	**10ms**	**50-150ms**	Mechanical arm movement dominates. Mostly replaced by SSD in modern systems.
Same data-center network	**—**	**1-5ms**	Pure network. No disk. Used for service-to-service calls inside one data center.
Database query (SSD-backed)	**—**	**10-50ms**	Includes all software overhead on top of SSD. This is your primary bottleneck.
Database query (no index)	**—**	**100ms-10s**	Full table scan. Reads every row. Catastrophic at scale. Always add indexes.
India to USA round trip	**—**	**200ms**	Speed of light limit through fiber. Cannot be optimised — must use CDN.
India to Europe round trip	**—**	**100-150ms**	Shorter distance. Still requires CDN for acceptable user experience.

Memory Anchor — The Ladder of Speed

CPU Cache → 0.0005ms → Lightning bolt ⚡
RAM/Cache → 0.1ms → Blink of an eye 👁
SSD (raw) → 0.1ms → Same as RAM hardware (but DB adds 10-50ms overhead)
DB Query → 10-50ms → Noticeable pause on slow Wi-Fi
India→USA → 200ms → Clearly perceptible delay — always use CDN for this
Full scan → 100ms-10s → Unacceptable in production — always add indexes

Why This Drives Component Choices

These numbers directly explain why each component exists:

COMPONENT	WHY IT EXISTS (backed by latency numbers)
Cache (Redis)	DB query costs 10-50ms. Cache costs 0.1-1ms. If SLA is 20ms, cache is not optional — it is mandatory.
CDN	India → USA costs 200ms per round trip. CDN edge server in Mumbai costs 5ms. For static files, CDN gives 40x improvement.
Read Replicas	One DB handles ~1000 queries/sec before degrading. At 5000 req/sec, add 4 read replicas so each handles 1000.
Message Queue	If a background job takes 500ms, putting it in a queue makes the user's response instant. They never wait for it.
Indexes	No-index query on 1B rows: full table scan (seconds). B-tree index on same table: 10-50ms. Non-negotiable at scale.

SECTION 2 — SCALE THRESHOLDS

These thresholds tell you WHEN to add each solution. The golden rule: start simple. Add complexity only when you hit the actual limit — not before. Over-engineering for scale you do not have is a common Senior Engineer mistake. Principal Engineers know when NOT to scale.

The Scaling Ladder — Start at the Bottom

STAGE	WHAT TO ADD AND WHY
< 1,000 users ~10 req/sec	Single server + single database. Nothing else. Any added complexity is waste. Most startups live here.
< 10,000 users ~100 req/sec	Add a Cache (Redis). Same queries hitting DB repeatedly — pre-store results. DB reads drop by 80%.
< 100,000 users ~1,000 req/sec	Add Read Replicas to DB (reads go to replicas, writes to primary). Add CDN for static files (explained below).
< 1,000,000 users ~10,000 req/sec	Shard your database horizontally. One DB cannot handle this write volume. Split data across multiple DB nodes.
< 10,000,000 users ~100,000 req/sec	Microservices architecture. Split monolith by domain (user service, feed service, payment service). Independent scaling.
< 100,000,000 users ~1,000,000 req/sec	Multi-region deployment. Active-Passive or Active-Active. Global load balancing. Edge computing at CDN layer.
> 100,000,000 users ~1M+ req/sec	Custom solutions. Google builds custom hardware. Facebook builds custom networking. Off-the-shelf stops working.

What is CDN for Static Files? — Fully Explained

'Add CDN for static files' is mentioned at the 100K user threshold. Here is exactly what it means:

STATIC FILES — Same for Every User

- Your app's logo (same for everyone)
- CSS stylesheets (same for everyone)
- JavaScript files (same for everyone)
- A viral celebrity photo (same for all 50M viewers)
- Product images in a catalogue
- → PERFECT for CDN — store one copy near each user

DYNAMIC CONTENT — Unique Per User

- Your personal Instagram feed
- Your bank account balance
- Your private messages
- Search results for your specific query
- Your personalised recommendations
- → CANNOT cache in CDN — different for every user

How CDN for Static Files Actually Works:

WITHOUT CDN: User in Mumbai requests Instagram's logo → request travels to Instagram's server in Virginia, USA (200ms round trip) → logo sent back. This happens for EVERY user, EVERY page load.

WITH CDN: Instagram uploads their logo ONCE to Cloudflare/Akamai CDN. CDN replicates it to 200+ edge servers worldwide including Mumbai.

Now: User in Mumbai requests logo → nearest CDN edge server in Mumbai responds (5ms) → 40x faster.

Your main server is now free: It only handles dynamic, personalised requests. Static files never touch it.

Cost saving: Static files are 60-80% of all web traffic by volume. CDN offloads all of that from your servers.

Rule of thumb: If the same file is requested by more than 1 user → it belongs on CDN.

Multi-Region: Active-Passive vs Active-Active — Fully Explained

At 100M+ users, one data centre in one country is not enough. You need servers in multiple geographic regions. The question is: how do those regions work together?

ACTIVE-PASSIVE (Simpler)

- Region 1 (India) = ACTIVE — handles ALL traffic
- Region 2 (USA) = PASSIVE — idle, just waiting
- If India goes down: traffic switches to USA
- Switchover takes 1-5 minutes (not instant)
- Cost: pay for 2 regions but only use 1
- Used by: smaller companies, internal systems
- Interview trigger word: 'disaster recovery'

ACTIVE-ACTIVE (Harder, More Expensive)

- Region 1 (India) = ACTIVE — handles Indian users
- Region 2 (USA) = ACTIVE — handles US users simultaneously
- If India goes down: USA instantly takes ALL traffic
- Zero switchover delay — no downtime at all
- Cost: both regions fully utilised (efficient)
- Challenge: data must sync between regions in real-time
- Used by: Google, Netflix, Visa, any global product

Why Active-Active is Hard — The Data Sync Problem:

If a user in India updates their profile, the US region must see that update immediately.

If two users in different regions update the same data simultaneously — which update wins?

This is the 'distributed data consistency' problem. It requires sophisticated conflict resolution.

Solutions: CRDTs (Conflict-free Replicated Data Types), vector clocks, last-write-wins policies.

This is why most companies start with Active-Passive and only move to Active-Active at Google/Netflix/Visa scale.

In interviews: mention Active-Passive first. Offer Active-Active as a scaling upgrade with the caveat that it requires solving data consistency across regions.

SECTION 3 — STORAGE NUMBERS & BACK-OF-ENVELOPE MATH

Back-of-envelope calculation is a core interview skill. Interviewers are not checking if your number is exactly right. They are checking if you can reason quantitatively and derive a sensible estimate quickly. Round aggressively. Show your work out loud.

Unit Reference — Never Get Confused Again

UNIT	WHAT IT EQUALS — IN HUMAN TERMS
1 Byte	8 bits. One single character: 'a', '5', '@'
1 Kilobyte (KB)	1,000 bytes. A short text message (160 chars) ≈ 160 bytes ≈ 0.16 KB
1 Megabyte (MB)	1,000 KB. A typical smartphone photo = 2-5 MB. A 3-minute MP3 song ≈ 3 MB
1 Gigabyte (GB)	1,000 MB. A full HD movie ≈ 1-4 GB. A DVD = 4.7 GB
1 Terabyte (TB)	1,000 GB. A typical laptop hard drive. ~500,000 photos.
1 Petabyte (PB)	1,000 TB. Facebook stores 100+ PB. Google processes 20+ PB per day.
1 Exabyte (EB)	1,000 PB. The entire internet is estimated at a few thousand exabytes.

Common Object Sizes — Memorise These

OBJECT	SIZE — AND WHY IT MATTERS
1 ASCII character	1 byte. Simple English text.
1 Unicode character	2-4 bytes. Supports all languages including Hindi, Chinese, Arabic.
1 tweet (max 280 chars)	~280 bytes text only. With metadata (user ID, timestamp, location, like count): ~500-600 bytes.
1 SMS text message	~160 bytes. This is why original SMS had 160 character limits.
1 UUID / unique ID	36 bytes as string ('550e8400-e29b-41d4-a716-446655440000'). 16 bytes as binary.
1 typical DB row	~1 KB (includes all fields, overhead). Use this as default if unsure.
1 small profile photo	~50-100 KB (compressed JPEG thumbnail)
1 typical profile photo	~200-500 KB (displayed size on app)
1 high-res photo	~2-8 MB (original from camera)
1 minute HD video (720p)	~60-100 MB
1 minute 4K video	~350-400 MB
1 minute audio (MP3)	~1 MB

The Tweet Storage Calculation — Fully Explained

This was asked in the previous session. Here is the complete step-by-step calculation:

MATH *Tweet Storage*	`500,000,000 tweets per day — Twitter's approximate daily volume` → **140 GB per day (text only) | ~275 GB per day (with full metadata)**

Full Calculation Walkthrough:

GIVEN: Twitter receives approximately 500 million tweets per day.

STEP 1 — Size of one tweet (text content only):
Maximum tweet length = 280 characters
1 character = 1 byte (ASCII English text)
1 tweet text = 280 bytes

STEP 2 — Daily storage for text only:
500,000,000 tweets × 280 bytes = 140,000,000,000 bytes
= 140,000,000 KB
= 140,000 MB
= 140 GB per day (text only)

STEP 3 — Real storage per tweet (with full metadata):
tweet_id (UUID): 16 bytes
user_id (integer): 8 bytes
content (text): 280 bytes
created_at (timestamp): 8 bytes
like_count (integer): 8 bytes
retweet_count: 8 bytes
location (lat/lng): 16 bytes
language code: 4 bytes
reply_to_tweet_id: 16 bytes
TOTAL per tweet: ~550-600 bytes (with DB overhead ~700-800 bytes)

STEP 4 — Daily storage with metadata:
500,000,000 × 700 bytes ≈ 350,000,000,000 bytes ≈ 350 GB per day

STEP 5 — 5-year total (for capacity planning):
350 GB/day × 365 days × 5 years ≈ 639 TB ≈ ~640 Terabytes

CONCLUSION: You need distributed storage — not one server. One HDD = 4-16 TB.
640 TB requires a cluster of 50-160 database servers minimum (just for tweets text).
This is why Twitter needs distributed systems — it is forced by the math.

SECTION 4 — BACK-OF-ENVELOPE TEMPLATES FOR INTERVIEWS

Use these templates in every system design interview. They take 5 minutes and immediately signal Principal-level quantitative thinking.

Template 1 — Traffic Estimation

The Formula:

Daily Active Users (DAU) = Total Users × Active Percentage
Requests per second (RPS) = DAU × Requests per user per day ÷ 86,400 seconds

TRICK: 86,400 seconds/day ≈ round to 100,000 for easy mental math
So: RPS ≈ DAU × Requests per user ÷ 100,000

Example — Design Twitter:
Total registered users: 500 million
Daily Active Users (DAU): 500M × 20% = 100 million
Requests per user per day: 20 (10 feed loads + 5 searches + 5 profile views)
Total requests per day: 100M × 20 = 2,000 million = 2 billion
Requests per second (avg): 2,000,000,000 ÷ 100,000 = 20,000 RPS
Peak (3x average): 20,000 × 3 = 60,000 RPS
Read:Write ratio: 100:1 (most users read, few tweet)
Write RPS: 60,000 ÷ 100 = 600 writes/sec
Read RPS: 60,000 - 600 ≈ 59,400 reads/sec

CONCLUSION: 60K RPS → needs load balancer + cache + read replicas. Cannot use single DB for reads.

Template 2 — Storage Estimation

The Formula:
Daily Storage = Events per day × Size per event
Total Storage = Daily Storage × Retention period (days)

Example — Design Twitter (storage for tweets):
New tweets per day: 500 million
Size per tweet: 700 bytes (text + metadata, as calculated above)
Daily storage: 500M × 700 bytes = 350 GB/day
Retention period: 5 years = 5 × 365 = 1,825 days
Total tweet storage: 350 GB × 1,825 = 638,750 GB ≈ 640 TB

Add media (photos/videos) — separate from DB, goes to Object Storage (S3):
10% of tweets have a photo: 50M photos/day × 200 KB = 10,000 GB = 10 TB/day
5% of tweets have a video: 25M videos/day × 5 MB = 125,000 GB = 125 TB/day
Total media per day: ~135 TB/day
Total media over 5 years: 135 TB × 1,825 ≈ 246,375 TB ≈ 246 Petabytes

CONCLUSION: Text tweets → distributed SQL/NoSQL cluster (~640 TB)
Media files → Object Storage like S3 (~246 PB over 5 years)
These are stored SEPARATELY. DB stores links to S3, not the files themselves.

Template 3 — Bandwidth Estimation

The Formula:
Ingress bandwidth = Write RPS × Average write payload size
Egress bandwidth = Read RPS × Average read response size

Example — Design Twitter (bandwidth):
Write RPS: 600 writes/sec
Write payload: 700 bytes per tweet
Ingress bandwidth: 600 × 700 = 420,000 bytes/sec ≈ 0.42 MB/sec (trivial)

Read RPS: 59,400 reads/sec

Read response: 20 tweets × 700 bytes = 14,000 bytes per response
Egress bandwidth: 59,400 × 14,000 = 831,600,000 bytes/sec ≈ 832 MB/sec ≈ 0.83 GB/sec

CONCLUSION: Writes are tiny. Reads dominate bandwidth completely.
0.83 GB/sec egress requires high-bandwidth network infrastructure and CDN offloading.
This confirms: Twitter is overwhelmingly READ-heavy. Optimise reads first. Always.

SECTION 5 — THE 3 QUESTIONS THAT DRIVE EVERY DESIGN DECISION

Before choosing any component — ask these 3 questions. They give you the quantitative basis for every decision. Interviewers score these questions more than the final diagram.

QUESTION	HOW TO USE IT
Question 1: What is the READ to WRITE ratio?	Read-heavy (100:1) → invest in cache, read replicas, CDN. Write-heavy (1:100) → invest in message queues, write sharding, append-only logs. Equal (1:1) → balance both. Example: payment systems.
Question 2: Does the user need to WAIT for this?	YES → synchronous. The response depends on the result. Example: posting a tweet needs a tweet ID back. NO → async via queue. Example: sending notifications, updating analytics, resizing images.
Question 3: What breaks FIRST under 10x load?	Usually: the database (disk I/O saturates first). Next: application servers (CPU/memory). Queues and caches scale horizontally very easily. This tells you where to invest in scaling first.

QUICK REFERENCE CARD — Memorise These Before Every Interview

Latency — The 5 Numbers That Matter

OPERATION	APPROXIMATE TIME
Cache read (Redis)	0.1 - 1 ms
Database query (with index, SSD)	10 - 50 ms
Database query (NO index — full scan)	100ms to 10 seconds — NEVER acceptable in production
Same data centre network call	1 - 5 ms
India to USA round trip	~200 ms — always use CDN for any static content crossing this

Scale — When to Add Each Solution

USERS / RPS	SOLUTION TO ADD
< 1K users / ~10 RPS	Nothing. Single server + single DB. Stay simple.
< 10K users / ~100 RPS	Add Cache (Redis). Eliminate repeated DB queries.
< 100K users / ~1K RPS	Add Read Replicas + CDN for static files.
< 1M users / ~10K RPS	Shard the database. Multiple DB nodes.
< 10M users / ~100K RPS	Microservices. Split by domain. Independent scaling.
< 100M users / ~1M RPS	Multi-region. Active-Passive → Active-Active.

Object Sizes — The 6 You Must Know

OBJECT	SIZE
1 character (ASCII)	1 byte
1 tweet (text only, max 280 chars)	~280 bytes text | ~700 bytes with full metadata
1 typical DB row (any table)	~1 KB (use as default when unsure)
1 compressed profile photo	~200 KB
1 minute of HD video	~100 MB
1 minute of 4K video	~375 MB

The Useful Approximations

APPROXIMATION	WHEN TO USE IT
86,400 sec/day ≈ 100,000	Converting daily events to per-second rate. Divide by 100K.
1 million requests/day ≈ 12 RPS	Quick DAU → RPS mental math. Divide millions by 100K.
1 billion = 10^9	Scale check: if your number hits billions, you need distributed systems.
3x rule for peak traffic	Average RPS × 3 = peak RPS. Design for peak, not average.

80/20 rule for cache	80% of traffic hits 20% of data. Cache that 20%. Hit rate will be ~80%.